# Solution: Maximum Number of Balloons (easy)

## Problem Statement

Given a string, determine the maximum number of times the word "balloon" can be formed using the characters from the string. Each character in the string can be used only once.

**Examples**:

1. **Example 1**:
   - Input: "balloonballoon"
   - Expected Output: 2
   - Justification: The word "balloon" can be formed twice from the given string.
2. **Example 2**:
   - Input: "bbaall"
   - Expected Output: 0
   - Justification: The word "balloon" cannot be formed from the given string as we are missing the character 'o' twice.
3. **Example 3**:
   - Input: "balloonballoooon"
   - Expected Output: 2
   - Justification: The word "balloon" can be formed twice, even though there are extra 'o' characters.

## Solution

To solve this problem, you start by creating a hashmap to count the frequency of each letter in the given string. Since the word "balloon" contains specific letters with varying frequencies (like 'l' and 'o' appearing twice), you need to account for these in your hashmap. Once you have the frequency of each letter, the next step is to determine how many times you can form the word "balloon". This is done by finding the minimum number of times each letter in "balloon" appears in the hashmap. The limiting factor will be the letter with the minimum frequency ratio to its requirement in the word "balloon". This approach ensures a balance between utilizing the available letters and adhering to the letter composition of "balloon".

1. **Character Frequency Count**: Traverse the string and populate a hashmap with the frequency count of each character.
2. **Determine Maximum Count**: Check the hashmap to determine the maximum number of times the word "balloon" can be formed. For characters 'b', 'a', and 'n', their frequency in the hashmap directly gives the number of times they can be used. For 'l' and 'o', we need to divide their frequency by 2.
3. **Result Calculation**: The minimum value among the counts of 'b', 'a', 'l'/2, 'o'/2, and 'n' will give the maximum number of times the word "balloon" can be formed.
4. **Return the Result**: Return the calculated minimum value as the final result.

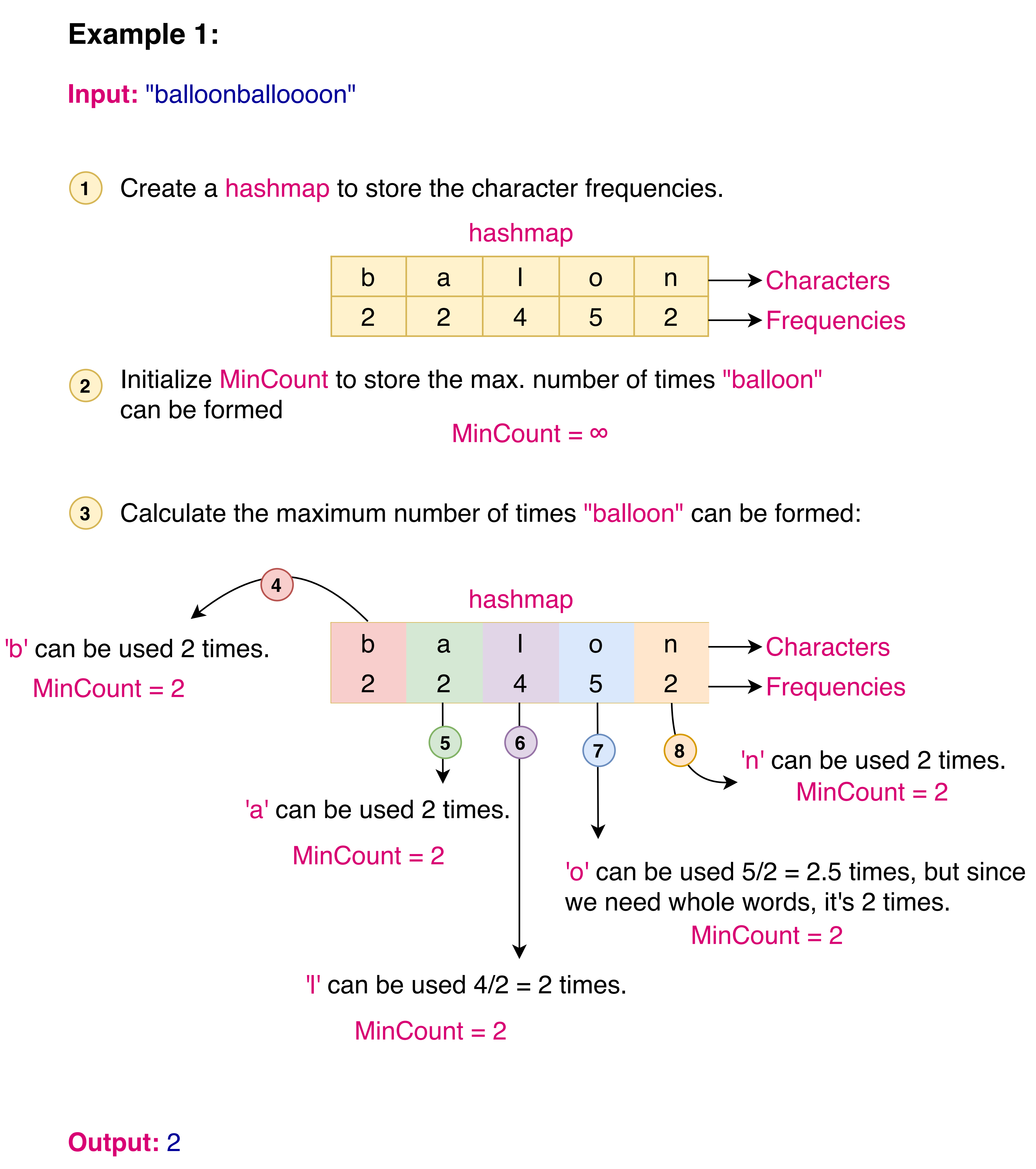
This approach is effective because it ensures that we account for the frequency of each character required to form the word "balloon". Using a hashmap allows for efficient storage and retrieval of character frequencies.

**Algorithm Walkthrough**:

Given the input string "balloonballoooon":

- Initialize an empty hashmap.
- Traverse the string and populate the hashmap with character frequencies: {'b':2, 'a':2, 'l':4, 'o':5, 'n':2}.
- Calculate the maximum number of times "balloon" can be formed:
  - 'b' can be used 2 times.
  - 'a' can be used 2 times.
  - 'l' can be used 4/2 = 2 times.
  - 'o' can be used 5/2 = 2.5 times, but since we need whole words, it's 2 times.
  - 'n' can be used 2 times.
- The minimum among these values is 2, which is the final result.

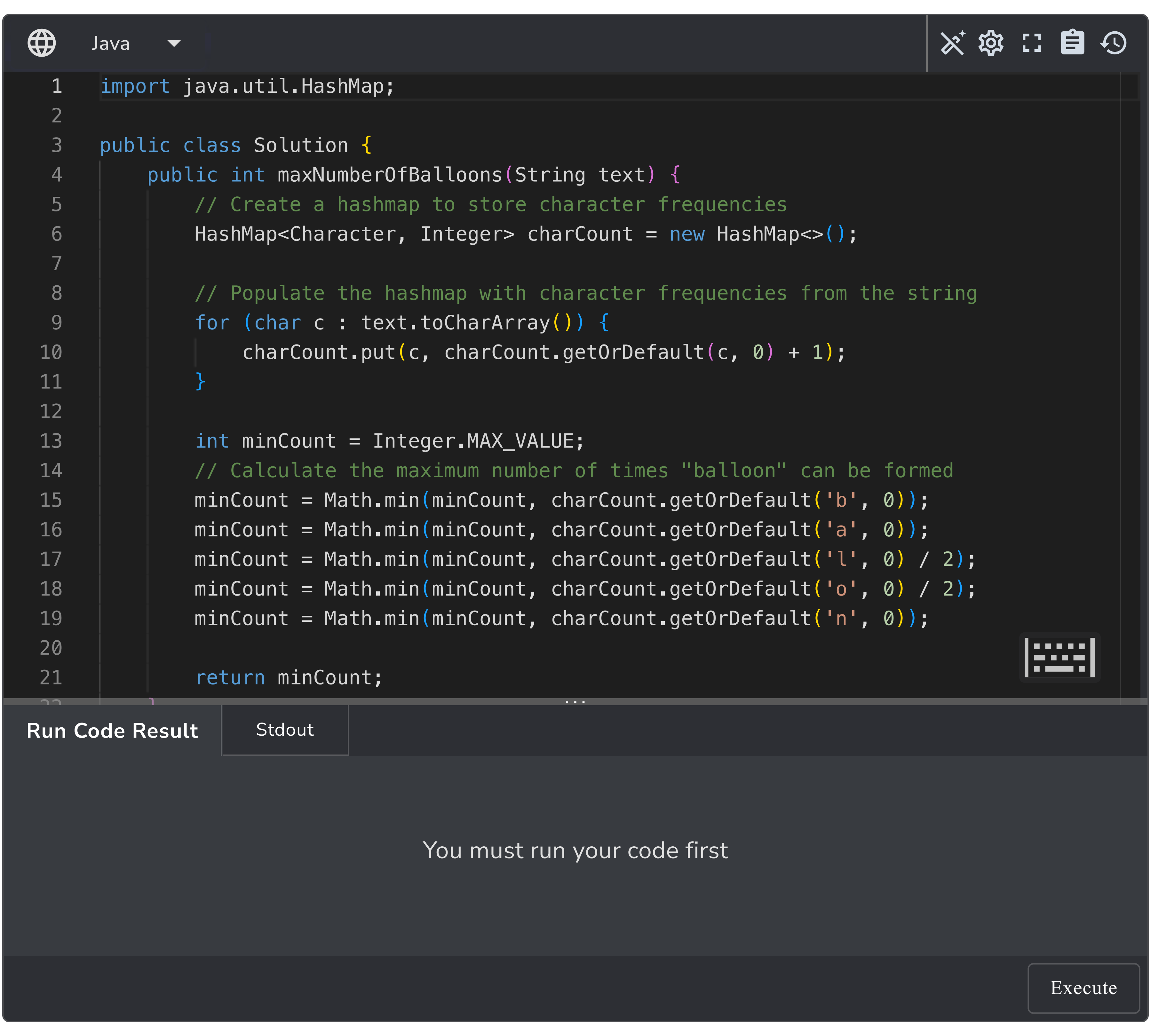
**Here is the visual representation of the algorithm**:


Example 1:
Input: "balloonballoooon"
1 Create a hashmap to store the character frequencies.
hashmap
b a l o n
2 2 4 5 2
Characters
Frequencies
2 Initialize MinCount to store the max. number of times "balloon" can be formed
MinCount = ∞
3 Calculate the maximum number of times "balloon" can be formed:
4
'b' can be used 2 times.
MinCount = 2
5
'a' can be used 2 times.
MinCount = 2
6
'l' can be used 4/2 = 2 times.
MinCount = 2
7
'o' can be used 5/2 = 2.5 times, but since we need whole words, it's 2 times.
MinCount = 2
8
'n' can be used 2 times.
MinCount = 2
Output: 2

Max. Number of Balloons

## Code

Here is the code for this algorithm:


Java
Run Code Result
Stdout
Execute

```java
import java.util.HashMap;

public class Solution {
    public int maxNumberOfBalloons(String text) {
        // Create a hashmap to store character frequencies
        HashMap<Character, Integer> charCount = new HashMap<>();

        // Populate the hashmap with character frequencies from the string
        for (char c : text.toCharArray()) {
            charCount.put(c, charCount.getOrDefault(c, 0) + 1);
        }

        int minCount = Integer.MAX_VALUE;
        // Calculate the maximum number of times "balloon" can be formed
        minCount = Math.min(minCount, charCount.getOrDefault('b', 0));
        minCount = Math.min(minCount, charCount.getOrDefault('a', 0));
        minCount = Math.min(minCount, charCount.getOrDefault('l', 0) / 2);
        minCount = Math.min(minCount, charCount.getOrDefault('o', 0) / 2);
        minCount = Math.min(minCount, charCount.getOrDefault('n', 0));

        return minCount;
```

You must run your code first

## Complexity Analysis

**Time Complexity**: The algorithm traverses the string once to populate the hashmap, which is O(n), where n is the length of the string. The subsequent operations are constant time. Therefore, the overall time complexity is O(n).

**Space Complexity**: The space complexity is determined by the hashmap, which in the worst case will have an entry for each unique character in the string. However, since the English alphabet has a fixed number of characters, the space complexity is O(1).

← Previous
Problem 3: Maximum Number Of Balloons (Easy)

Next →
Problem 4: Longest Palindrome(Easy)

☐ Mark as Completed